

EduCal

Academic Planning Application

Daniel Santos

Orientadores: Filipe Freitas
Sandra Ferreira

Relatório do projeto realizado no âmbito de Projeto
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

EduCal

Academic Planning Application

51701 Daniel Rodrigues Santos

Orientadores: Filipe Bastos de Freitas

Sandra Cristina Pereira Ferreira Neves

Relatório do projeto realizado no âmbito de Projeto
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Julho de 2026

Abstract

ISEL’s academic services currently plan recurring institutional activities, such as application periods, enrolments, exams, and their associated deadlines, using spreadsheets. This process requires the entire calendar to be manually rebuilt every academic year, with every date individually checked against weekends, holidays, and vacation periods, making it slow and prone to error. EduCal is a web application that replaces this manual process with a structured, rule-aware academic planning tool. Events are modeled hierarchically, with each high-level academic objective linked to one or more dated occurrences carrying institutional metadata such as category, status, and responsible department. Its central feature, the Migration Engine, transfers the calendar from one academic year to the next, idempotently cloning events and shifting their dates forward by one year, while a modular Business Rule Engine checks the result against configurable institutional constraints, namely weekends, national holidays, vacation periods, and minimum gaps between events, surfacing violations for manual review rather than silently correcting them. The system is built with Next.js and PostgreSQL, and secures access through a role-based authentication system (Viewer, Editor, Administrator). At the conclusion of the project, event management, the migration engine, the rule engine, and role-based access control are fully implemented and in use; notification templates and PDF schedule export, both part of the original scope, were not delivered.

Keywords: academic planning; calendar management; business rule engine; Next.js; PostgreSQL.

Resumo

Os serviços académicos do ISEL planeiam atualmente atividades institucionais recorrentes, como candidaturas, matrículas, exames e os respetivos prazos, recorrendo a folhas de cálculo. Este processo obriga a reconstruir manualmente todo o calendário em cada ano letivo, com cada data a ser verificada individualmente contra fins de semana, feriados e períodos de férias, tornando o processo lento e sujeito a erros. O EduCal é uma aplicação web que substitui este processo manual por uma ferramenta de planeamento académico estruturada e sensível a regras institucionais. Os eventos são modelados de forma hierárquica, em que cada objetivo académico de alto nível está associado a uma ou mais ocorrências datadas, com metadados institucionais como categoria, estado e responsável. A sua funcionalidade central, o Motor de Migração, transfere o calendário de um ano letivo para o seguinte, clonando eventos de forma idempotente e avançando as suas datas em um ano, enquanto um Motor de Regras de Negócio modular valida o resultado face a restrições institucionais configuráveis, nomeadamente fins de semana, feriados nacionais, períodos de férias e intervalos mínimos entre eventos, assinalando violações para revisão manual em vez de as corrigir silenciosamente. O sistema foi desenvolvido com Next.js e PostgreSQL, e protege o acesso através de um sistema de autenticação baseado em papéis (Visualizador, Editor, Administrador). À conclusão do projeto, a gestão de eventos, o motor de migração, o motor de regras e o controlo de acesso baseado em papéis encontram-se totalmente implementados e em uso; os templates de notificações e a exportação de calendários em PDF, ambos previstos no âmbito inicial, não foram entregues.

Palavras-chave: planeamento académico; gestão de calendário; motor de regras de negócio; Next.js; PostgreSQL.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	System Requirements	2
1.2.1	Functional Requirements	2
1.2.2	Non-Functional Requirements	3
1.3	Report Organization	3
2	Background and Problem Definition	5
2.1	Problem Statement	5
2.2	State of the Art	6
2.3	Objectives	7
3	System Architecture and Data Model	9
3.1	Global System Overview	9
3.2	Technology Stack	10
3.2.1	Backend	10
3.2.2	Frontend	10
3.3	Data Model and Entity-Relationship Diagram	10
3.3.1	Core Scheduling Entities	12
3.3.2	Classification and Management	12
3.3.3	Automation and Business Rules	12
3.3.4	Security and Auditing	13
3.4	Data Access Strategy	13
4	Backend	15
4.1	Organization	15
4.2	Authentication	16
4.3	Error Handling	17
4.4	Role-Based Access Control	17
4.5	Layers	17
4.5.1	Controller	17

4.5.2	Service	18
4.5.3	Data	18
4.6	Migration Engine	18
4.7	Business Rule Engine	19
4.8	Tests	21
5	Frontend	23
5.1	Structural Organization	23
5.2	API Communication Architecture	23
5.3	Web Application	24
5.3.1	Navigation and Access Control	24
5.3.2	Authentication	24
5.3.3	Calendar Interface	25
5.3.4	Event Management Dialogs	27
5.3.5	Academic Year Tools	29
5.3.6	User Management Dashboard	30
6	Deployment and Infrastructure	31
6.1	Current Infrastructure	31
6.2	Deployment Roadmap	31
7	Conclusion	33
	References	35

List of Figures

3.1	System architecture diagram.	9
3.2	Entity-Relationship Diagram, derived from the Drizzle schema.	11
5.1	Login interface.	25
5.2	Main calendar interface (Month view).	27
5.3	Add/Edit Event dialog.	28
5.4	Event Details dialog.	28
5.5	Validation/migration results modal.	29
5.6	Vacation periods dialog.	30
5.7	User management dashboard.	30

List of Code Examples

4.1	Calendar events API route.	15
4.2	Password hashing and verification.	16
4.3	Date shifting with leap-year clamping.	19
4.4	The Weekend rule definition.	20
5.1	The DraggableEvent wrapper component.	25

Chapter 1

Introduction

As part of the Bachelor’s Degree in Computer Science, Networks, and Telecommunications Engineering, this report details the development of the final course project entitled EduCal, an Academic Planning Application. The project, carried out during the 2025-2026 academic year, arises from the need to optimize the management of institutional services that involve critical planning, execution, and improvement activities.

Currently, these activities take place on dates set annually. However, there is no tool to facilitate the transition from one calendar year to the next while respecting specific criteria and deadlines. The current scenario for planning annual tasks is based on dense and complex spreadsheets. Although detailed, these tools are prone to filling errors and require an exhaustive manual effort to replicate the network of objectives, requesters, and instructions in the new school year.

1.1 Motivation

The EduCal project aims to replace this manual process with a web-based application. Unlike general-purpose calendars, which are well suited for personal scheduling but lack specific institutional logic, EduCal is designed specifically for academic planning through several key differentiators.

Primarily, it uses a hierarchical event model where an “Event” represents a high-level academic objective linked to multiple independent “Occurrences,” allowing complex metadata, such as institutional classifications and assigned responsible personnel, to be attached to each entry. Furthermore, the project’s core functionality is the Migration Engine, which does not simply copy events from one year to the next, but clones the following year’s calendar and shifts each occurrence’s dates forward by one year. The engine does not itself correct dates that fall on a weekend, a national holiday, or a vacation period; instead, it runs the configurable business rules against the migrated calendar and reports any such violations for manual review. Additionally, EduCal includes institutional workflow metadata, such as planning status and responsibility tracking, to support the day-to-day management of the

calendar.

Given the nature of the project, its initial deployment is expected to take place within ISEL's services. To this end, the solution was developed using Next.js [1] and PostgreSQL [2], and designed to be hosted on the institution's internal network. The application implements a decoupled architecture that separates business logic from data storage, allowing the underlying storage technology to change without affecting the rest of the system.

1.2 System Requirements

To ensure the EduCal application effectively addresses the identified problems and achieves its core objectives, a detailed set of requirements was defined. These specifications served as the technical blueprint for the development process, categorizing the system's needs into functional capabilities and non-functional quality standards.

1.2.1 Functional Requirements

Functional requirements outline the specific features, tasks, and behaviors the platform must provide to satisfy user needs. These define exactly what the system does, ranging from the fundamental management of academic events to the automated execution of the migration engine.

- **RF01, Event and Occurrence Management:** the system must allow the creation and management of events that have multiple annual occurrences, each with its own descriptions and dates.
- **RF02, Migration Engine:** it must be possible to transfer the event history from one academic year to the next, automatically validating the resulting dates.
- **RF03, Rule Configuration:** the system must allow flexible business rules (e.g. working days, minimum gaps) to be defined, which the migration engine and event editor must respect.
- **RF04, Notification Templates:** the platform must allow the configuration of email or other text templates associated with events for sending proactive notifications.
- **RF05, Schedule Export:** the system must generate an academic calendar in PDF format for institutional distribution.
- **RF06, Profile Access Control:** multiple user roles must be supported to ensure data security, namely a Viewer with read-only access to consult planning and schedules, and an Editor/Administrator with full permissions to perform CRUD operations, configure rules, and run the migration engine.

1.2.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance criteria of the system. Rather than focusing on specific behaviors, they define how the system performs, ensuring that the application is secure, maintainable, and provides a good user experience.

- **RNF01, Data Integrity:** business days, holidays, and school vacation periods must be validated against the migration process, so that any resulting conflict is reported rather than passed through unnoticed.
- **RNF02, Usability:** the interface must be intuitive and capable of presenting the necessary density of information across multiple calendar views.
- **RNF03, Architecture and Scalability:** the system must be modular and scalable, ensuring fast access to extensive calendars and allowing storage technology to evolve independently of business logic.

1.3 Report Organization

The remainder of this report is organized as follows. Chapter 2 describes the problem being addressed, centered on the limitations of the spreadsheet-based process currently used by ISEL’s academic services. Chapter 3 presents the system architecture, including the technology stack and the Entity-Relationship data model. Chapter 4 details the backend implementation, covering its layered organization, authentication and access control mechanisms, the Migration Engine, and the Business Rule Engine. Chapter 5 describes every screen and dialog of the frontend, its structural organization, and how it communicates with the backend. Chapter 6 discusses the deployment strategy and current infrastructure. Finally, Chapter 7 summarizes which requirements were delivered and which were not, and closes the report.

Chapter 2

Background and Problem Definition

This chapter describes the institutional context that motivates the EduCal project, details the concrete problems found in the current process used by ISEL’s academic services, and states the objectives the project sets out to achieve.

2.1 Problem Statement

ISEL’s academic services are responsible for planning and coordinating a large number of recurring institutional activities every academic year, among others application periods, enrolment windows, exam periods, and the various administrative deadlines associated with them. Each of these activities is not a single, isolated event: it typically recurs across the year in multiple occurrences, is owned by a specific department or responsible party, and must respect a set of institutional constraints, such as not falling on a weekend, a national holiday, or a school vacation period.

Today, this planning is done using spreadsheets (Excel). While spreadsheets are flexible and familiar, they are not designed to encode this kind of institutional logic, which creates several concrete problems:

- **No structured relationship between events and their occurrences.** A spreadsheet row is just a row; it cannot natively express that a single academic objective (e.g. “Enrolment Period”) has several dated occurrences throughout the year, each carrying its own metadata. This relationship has to be maintained manually by whoever edits the sheet.
- **No automated year-to-year transition.** At the start of each academic year, the entire calendar has to be manually rebuilt from the previous year’s sheet: every date has to be individually recalculated and moved forward, checked against weekends, holidays, and vacation periods, and re-entered. There is no tool that performs this transition automatically.
- **No enforcement of business rules.** Spreadsheets do not validate that a date does

not fall on a weekend or holiday, that an event respects a minimum duration, or that two related events keep a minimum gap between them. These rules exist informally, in the knowledge of the people who plan the calendar, and are checked by hand, which makes them easy to miss.

- **Prone to filling errors and duplicated effort.** Because the network of objectives, responsible parties, and instructions has to be replicated by hand every year, the process is exhaustive and error-prone: a single incorrect date or a missed dependency can propagate through the rest of the plan.
- **No institutional workflow metadata.** Spreadsheets do not naturally support classifying events (e.g. by category, planning status, or responsible department) in a way that can be filtered, queried, or reported on; this metadata, if it exists at all, is scattered across free-text cells or separate documents.

These limitations directly motivate EduCal, a tool that models academic planning as structured data instead of free-form spreadsheet rows, so that year-to-year transitions, rule checking, and institutional metadata can be handled by software rather than by manual, error-prone repetition.

2.2 State of the Art

Before designing a purpose-built system, it is worth asking whether an existing, general-purpose calendar tool could already solve this problem. The two most used products in this space are Google Calendar and Microsoft Outlook/Calendar, both of which are mature, widely adopted, and already familiar to most staff at an institution such as ISEL.

Both tools are built for scheduling individual meetings and personal appointments, sharing calendars between people, sending invites with RSVP tracking, and booking shared resources such as meeting rooms. Both also support recurring events, generally based on the iCalendar RRULE standard (e.g. “every year on this date”, “every weekday”), which lets a single event repeat automatically according to a fixed pattern.

However, neither tool addresses the specific institutional problem described in Section 2.1:

- **No hierarchical event/occurrence model with institutional metadata.** A recurring event in Google Calendar or Outlook is just one event that repeats; it has no concept of an academic objective owning multiple independently-manageable occurrences, each carrying institution-specific metadata such as category, planning status, or a responsible department. Encoding this would require using free-text fields or external spreadsheets on top of the calendar, which is the exact problem EduCal sets out to remove.

- **No academic-year migration.** Standard recurrence rules repeat an event on a fixed pattern (e.g. the same date every year); they cannot re-derive a whole calendar of interdependent events for a new academic year while checking each one individually against that year's specific holidays and vacation periods. There is no equivalent of EduCal's Migration Engine.
- **No configurable institutional business rules.** Neither tool can validate, at the moment an event is created or migrated, whether it falls on a weekend, a national holiday, or a vacation period, or whether it keeps a minimum gap from another related event; these are the constraints central to EduCal's Business Rule Engine. Users would have to check these manually, which is precisely the error-prone behaviour observed today with spreadsheets.
- **No institution-specific access model.** Sharing permissions in these tools are generic (owner/editor/viewer of a calendar), not tied to an institutional role hierarchy with self-management invariants such as requiring at least one administrator to always remain (Section 4.4).

In short, Google Calendar and Outlook are well suited to personal and team scheduling, but they operate at the level of individual events, not at the level of an institutional planning process with hierarchical objectives, yearly migration, and configurable rule checking. This gap is what justifies building EduCal as a dedicated system rather than adopting an existing calendar product.

2.3 Objectives

EduCal aims to replace this manual, spreadsheet-based effort with a web application. The main objectives are:

- **Baseline Creation:** create an academic calendar from scratch, structured around events and their occurrences, as the foundation for the rest of the project's functionality.
- **Automation:** automatically migrate the academic calendar from the current year to the next, instead of rebuilding it by hand every year.
- **Application of Rules:** check compliance with configurable business rules, such as whether events fall on holidays, weekends, or outside school vacation periods, replacing the informal, manual checks used today.
- **Centralization:** unify the planning workflow in a single system that eliminates redundant spreadsheets and ensures data integrity and consistency across the institution.

Chapter 3

System Architecture and Data Model

To ensure the project’s viability, maintainability, and security, the architecture was designed around technologies that are widely adopted in the industry and that allow the application to be hosted within ISEL’s own infrastructure.

3.1 Global System Overview

EduCal follows a three-tier architecture that separates presentation, business logic, and data storage.

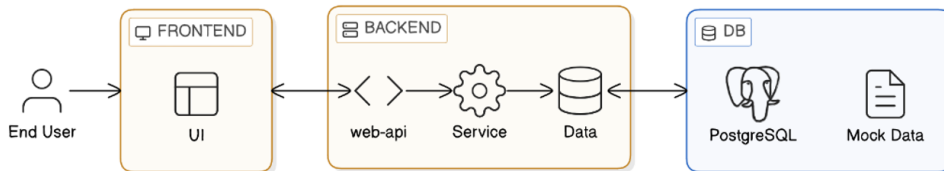


Figure 3.1: System architecture diagram.

As illustrated in Figure 3.1, the system is organized into three layers:

- **Presentation Layer (Frontend):** built with Next.js and React, it manages the state of the complex calendar views (Agenda, Day, Week, Month, and Year) and provides specialized interfaces for event and occurrence management, such as dynamic forms for multi-occurrence events.
- **Logic Layer (Backend):** implemented as Next.js API routes backed by a Controller-Service-Data structure (detailed in Chapter 4). The controllers act as the entry point, ensuring only authenticated users with the required role can modify the institutional calendar; the services hold the core business logic, including the Migration Engine (RF02)

and the Business Rule Engine (RF03); and the data layer abstracts all communication with the underlying storage.

- **Data Layer (DB):** a PostgreSQL relational database provides persistent storage for events, occurrences, and system configuration, accessed through the Drizzle ORM.

All components outlined in this architecture have been implemented, including PostgreSQL, so far tested only against a local instance (Chapter 6). The data access layer isolates all storage behind a single point in the code, so business services never talk to the database directly. This is what makes it possible to keep an alternative, JSON-based storage backend, currently used for the Vercel preview deployment, without any changes to the business logic, API routes, or frontend (see Section 3.4).

3.2 Technology Stack

3.2.1 Backend

The backend is built on **Next.js** [1] (v16, App Router), using its API routes as the HTTP entry point rather than a separate server framework. Data persistence uses **PostgreSQL** [2] through **Drizzle ORM**, a TypeScript-first ORM that provides both a relational query API and raw SQL migrations (**drizzle-kit**). Request payloads are validated end-to-end with **Zod**, including schemas that are built dynamically from live database values (e.g. valid event categories). Password hashing uses Node.js's built-in **crypto** module [3] (**scrypt**), and session cookies follow the **Secure/HttpOnly** guidance from the OWASP Session Management Cheat Sheet [4]. National holiday and business-day calculations use the **date-fns** and **date-holidays** libraries.

3.2.2 Frontend

The frontend is built with **React 19** inside the same Next.js application (no separate frontend project), styled with **Tailwind CSS v4** and the **shadcn/ui** component library (built on Radix UI primitives). Forms use **react-hook-form** together with Zod resolvers for validation, **sonner** provides toast notifications for asynchronous feedback, and **framer-motion** drives the calendar's drag-and-drop and transition animations. The project uses **pnpm** as its package manager and **TypeScript** throughout both the frontend and backend.

3.3 Data Model and Entity-Relationship Diagram

To support automated year-to-year migration and rule validation, a relational database schema was designed and implemented with Drizzle ORM, visually outlined in the Entity-Relationship Diagram in Figure 3.2, generated directly from the Drizzle schema definitions (**db/schema/auth.schema.ts**, **db/schema/calendar.schema.ts**, **db/schema/relations.ts**).

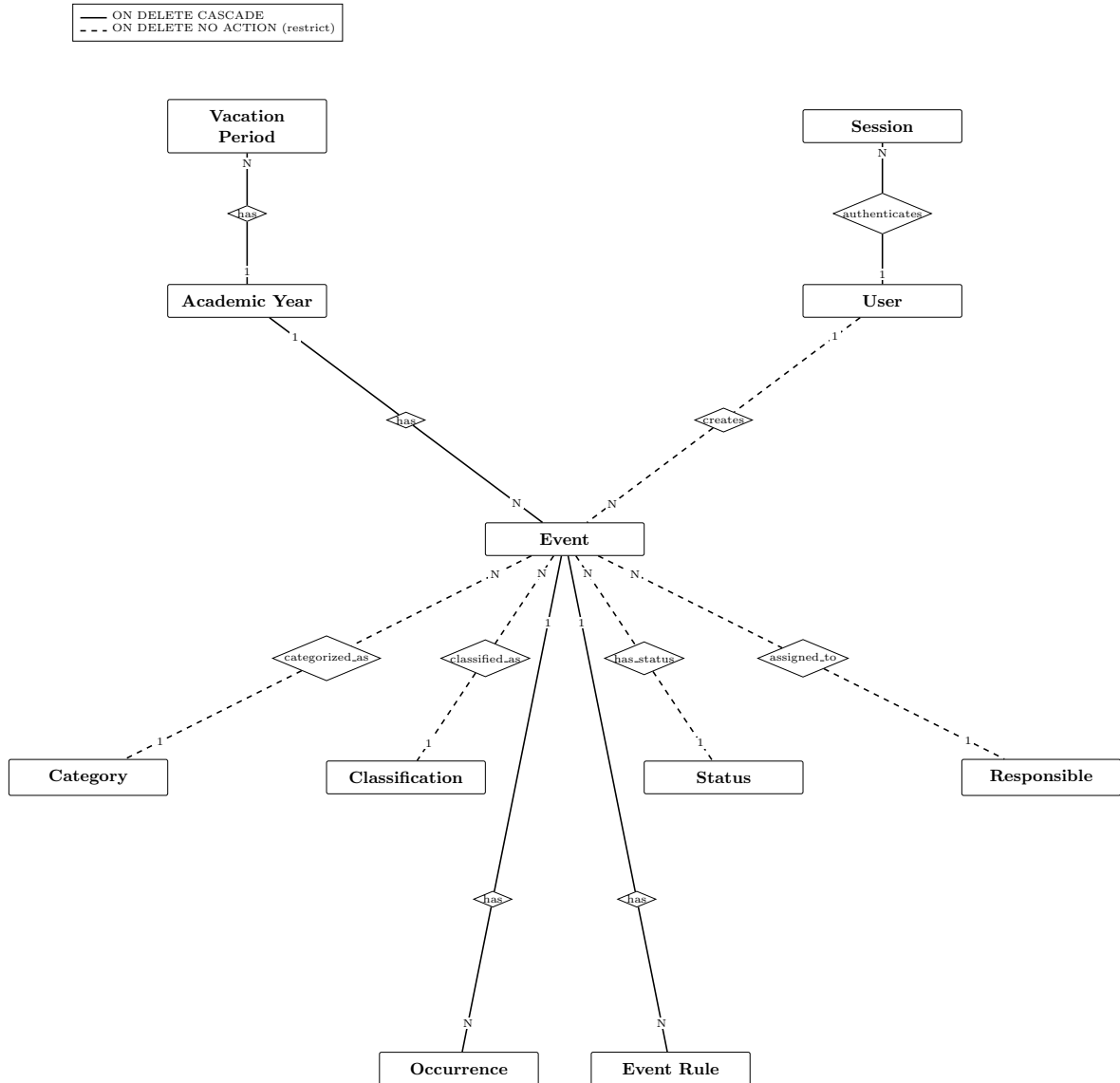


Figure 3.2: Entity-Relationship Diagram, derived from the Drizzle schema.

Each diamond names the relationship between the two entities it connects, with cardinalities (1/N) marked at each end. Solid lines are foreign keys with `ON DELETE CASCADE`; dashed lines are `ON DELETE NO ACTION` (the referenced row cannot be deleted while still in use).

3.3.1 Core Scheduling Entities

The system’s core logic centers on the relationship between events and occurrences, plus the `academic_years` table that anchors both to a specific year:

- **Academic Year:** keyed by its start year, stores a label and the concrete `start_date/end_date` that bound the academic year.
- **Event:** stores high-level metadata such as the objective, category, classification, status, responsible party, and the academic year and user that own it.
- **Occurrence:** a concrete instance of an event, with its own `start_date` and `end_date`, and optional `min_days/min_days_to_next` constraints. Each event can have multiple occurrences, and every occurrence is deleted automatically (`ON DELETE CASCADE`) when its parent event is deleted.

3.3.2 Classification and Management

To facilitate filtering and institutional reporting, the system uses four normalization/lookup tables referenced by events: **Category** (which also stores a `color`, used to render the category badges and filter dots in the calendar UI, Section 5.3.3), **Classification**, **Status**, and **Responsible**. Each event references one value from each of these tables, allowing events to be categorized (e.g. “TFM”, “Candidaturas”), tracked through a planning workflow status (e.g. “Por Fazer”, “Em Curso”, “Feito”), and attributed to an owning department (e.g. “Serviços Acadêmicos”, “Conselho Pedagógico”). Unlike the scheduling entities above, these reference tables are not cascade-deleted: a category, classification, status, or responsible value that is still referenced by an event cannot be removed, which prevents institutional data from silently losing its classification.

3.3.3 Automation and Business Rules

Each event can have zero or more **Event Rules** attached (`event_rules` table), storing a rule `type` and a JSON `config` object. These rows are what the Business Rule Engine evaluates for that event (Section 4.7); the **Vacation Period** table, scoped to an academic year, stores the school vacation periods that both the rule engine and the migration engine consult.

3.3.4 Security and Auditing

The **User** and **Session** entities manage multi-role access control. The User table stores a **script**-hashed password and a role (**viewer**, **editor**, or **admin**); the Session table tracks active session tokens and their expiration, and is deleted automatically when its owning user is deleted. Every Event additionally stores the **user_id** of its creator, providing a basic audit trail of who owns each entry in the calendar.

3.4 Data Access Strategy

The data access layer isolates all storage behind a repository interface per feature (**IAuthRepository**, **ICalendarRepository**), so that business services only ever call the interface, never a concrete database client. Two concrete implementations exist for each interface: a PostgreSQL implementation built on Drizzle ORM (**AuthRepositoryPostgres**, **CalendarRepositoryPostgres**), intended for the institutional VM deployment (Chapter 6), and a JSON-backed implementation on top of Upstash Redis (**AuthRepositoryRedis**, **CalendarRepositoryRedis**), used for the Vercel preview deployment and local development, since it requires no remote database server to be provisioned.

Which implementation is used is decided by a small factory, selected through the **DATA_LAYER** environment variable: any value other than the literal **"redis"** (including the variable being unset) resolves to PostgreSQL. Because every business service depends only on the repository interface, switching between the two backends requires no change to controllers, services, API routes, or the frontend; only the environment variable changes. This design was validated in practice during development, when the project moved from the JSON-backed mock data used while building the business logic to a locally-tested PostgreSQL database without touching the migration engine, the rule engine, or any UI code.

Chapter 4

Backend

This chapter details the backend implementation of EduCal, covering its layered organization, authentication, authorization, error handling, and testing approach.

4.1 Organization

The backend is organized per feature (`auth`, `calendar`) under `server/`, each following the same three-layer structure of Controller, Service, and Data, plus a `server/shared/` module for cross-cutting concerns (the API response envelope, domain error type, and the data-layer factory). This mirrors the Controller-Service-Repository pattern used since the initial implementation: controllers such as `authController` and `calendarController` act as orchestrators that parse and validate the incoming HTTP request and hand off to the corresponding service, keeping the route handlers in `app/api/**/*.route.ts` as thin wrappers, as shown in Code Example 4.1.

```
1 export const GET = withApiAuth(async () => {  
2   return calendarController.listEvents();  
3 });  
4  
5 export const POST = withApiAuth(async (request) => {  
6   return calendarController.createEvent(request);  
7 });
```

Code Example 4.1: Calendar events API route (`app/api/events/route.ts`): each HTTP verb is wired directly to a controller method, behind the authentication middleware.

Neither handler parses the request itself, checks the role, or catches errors: `withApiAuth` (Section 4.2) resolves the session cookie into a user before either handler runs, and every failure, whether it comes from validation, authorization, or the database, is funnelled through the controller's own `fail()` call (Section 4.3). This keeps every route file in `app/api/` at a near-identical, one-line shape, regardless of which feature or HTTP verb it implements.

4.2 Authentication

Passwords are never stored in plain text: they are hashed with Node.js’s built-in `crypto` module [3] using `scrypt`, a per-password random 16-byte salt, and a 64-byte derived key, stored as `scrypt$salt$hash`. Password verification uses a constant-time comparison (`crypto.timingSafeEqual`) to reduce the risk of timing attacks. Code Example 4.2 shows the implementation of both functions.

```
1 export const hashPassword = (password: string): string => {
2   const salt = crypto.randomBytes(16).toString("hex");
3   const hash = crypto.scryptSync(password, salt, 64).toString("hex");
4   return `scrypt${salt}${hash}`;
5 };
6
7 export const verifyPassword = (password: string, encodedHash: string): boolean => {
8   const [algorithm, salt, storedHash] = encodedHash.split("$");
9   if (algorithm !== "scrypt" || !salt || !storedHash) {
10    return false;
11  }
12
13  const computedHash = crypto.scryptSync(password, salt, 64).toString("hex");
14  if (computedHash.length !== storedHash.length) return false;
15  return crypto.timingSafeEqual(
16    Buffer.from(storedHash, "hex"),
17    Buffer.from(computedHash, "hex"),
18  );
19 };
```

Code Example 4.2: Password hashing and verification (`server/auth/crypto.ts`).

`hashPassword` generates a fresh random salt for every password and derives a 64-byte key with `scryptSync`, encoding the algorithm tag, salt, and derived key into the single `$`-delimited string that is stored in the `users.password_hash` column. `verifyPassword` parses that string back into its three parts, rejecting anything not tagged `scrypt` or missing a segment, then re-derives the hash from the candidate password using the stored salt and compares it against the stored hash with `crypto.timingSafeEqual`. Comparing the two hashes’ lengths before calling `timingSafeEqual` avoids that function throwing on mismatched buffer sizes, while still not leaking any timing information for hashes of equal length.

On login, the service generates a random 48-byte, base64url-encoded session token and persists it, together with its expiration, in the `sessions` table. Two expiration policies are supported: a short-lived session (1 day) by default, or a 30-day session when the user selects “Remember Me” (`staySignedIn`). The session token is sent to the browser as a cookie that is `HttpOnly` (not accessible to client-side scripts, mitigating XSS-based token theft), `Secure` in production, and `SameSite=Lax` [4]; the cookie itself only carries a `maxAge` when “Remember Me” is selected, otherwise it is a session-only cookie. On every authenticated request, the session is also validated server-side against its stored expiration, and expired sessions are deleted rather than just ignored.

4.3 Error Handling

API responses follow a consistent envelope: successful responses return `{ data: ... }`, and failures return `{ error: { code, message, details } }`. Domain-level failures are raised as a typed `DomainError` (with an error code such as `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, or `CONFLICT`, and an HTTP status), and Zod validation failures are caught and converted into a `VALIDATION_ERROR` response with a structured, field-level breakdown of what failed. Every controller method funnels its errors through this single `fail()` helper, which guarantees the frontend always receives a predictable error shape to parse and display, regardless of which layer raised the error.

4.4 Role-Based Access Control

Three roles are supported, `viewer`, `editor`, and `admin`, ordered in a strict hierarchy. Calendar-editing operations (creating, updating, or deleting events, occurrences, and vacation periods, and running the migration engine) require at least the `editor` role; user management requires the `admin` role. Authentication itself is enforced by HTTP-level middleware (`withApiAuth/withApiAuthContext`) that resolves the session cookie into a user and rejects unauthenticated requests before they reach a controller; role checks are then applied one layer down, inside the services.

Beyond the basic role hierarchy, the system enforces three self-management invariants to prevent an institution from locking itself out of administration:

- an administrator cannot demote their own account away from `admin`;
- an administrator cannot delete their own account;
- the last remaining administrator cannot be demoted or deleted; the service counts existing admins before allowing either operation and rejects it if only one would remain.

4.5 Layers

4.5.1 Controller

Controllers (`server/auth/controllers/`, `server/calendar/controllers/`) are the entry point for each API route. They parse and validate route parameters and the request body, invoke the corresponding service method, and translate the result, or any error thrown, into an HTTP response using the shared `ok()/fail()` helpers. This is also where the session cookie is set on login and cleared on logout.

4.5.2 Service

Services (`server/auth/services/auth.service.ts`, `server/calendar/services/calendar.service.ts`) hold all business logic and authorization checks, and are the bridge between the controllers and the data layer. This is where the RBAC invariants described in Section 4.4 are enforced, and where the two central pieces of calendar business logic live: the Migration Engine (Section 4.6) and the Business Rule Engine (Section 4.7).

4.5.3 Data

The data layer (`server/auth/data/`, `server/calendar/data/`) exposes a repository interface (`IAuthRepository`, `ICalendarRepository`) per feature, abstracting all storage access. Services depend only on this interface, never on a concrete database client, which is what enables the pluggable PostgreSQL/Redis backend described in Section 3.4.

4.6 Migration Engine

The migration of the academic calendar from one year to the next is implemented as a single service operation (`CalendarService.migrateAcademicYear`), exposed to editors and administrators through dedicated “Validate” and “Migrate” actions in the calendar header (Section 5.3.5).

- **Vacation periods first:** for the target academic year (source year + 1), the engine migrates vacation periods before events, shifting each period’s dates forward by one year so that subsequent rule checks against the target year already see the correct vacation windows.
- **Idempotent event migration:** every source-year event is fingerprinted from a fixed subset of its fields (name, objective, category, classification, status, responsible, owner, and each occurrence’s description and dates, deliberately excluding database-generated ids). If an equivalent event already exists in the target year, it is skipped; otherwise it is cloned and inserted. This makes the operation safe to run more than once without creating duplicates.
- **Date shifting and leap years:** each occurrence’s dates are shifted forward by one calendar year. The 29th of February is handled explicitly: if shifting the year would otherwise roll the date over into March (because the target year is not a leap year), the date is instead clamped back to the 28th of February.
- **Validation, not automatic correction:** immediately after an event is migrated, the full business rule set (Section 4.7) is run against it, and any violations, such as an occurrence now falling on a weekend or holiday, are collected into a structured

report returned to the caller. The engine does not shift dates to avoid these conflicts; resolving them is left as a manual, user-driven step, guided by the issue report shown in the application (Section 5.3.5). The same rule-checking logic also powers the standalone “Validate” action, which runs a read-only check without migrating anything.

Code Example 4.3 shows `shiftDateByYears`, the pure function every occurrence’s start and end dates are passed through during migration, which implements the leap-year clamping described above.

```
1 export const shiftDateByYears = (dateValue: string, years: number): string => {
2   const source = new Date(dateValue);
3   const target = new Date(dateValue);
4   target.setUTCFullYear(source.getUTCFullYear() + years);
5
6   if (target.getUTCMonth() !== source.getUTCMonth()) {
7     target.setUTCDate(0);
8   }
9
10  return target.toISOString();
11 };
```

Code Example 4.3: Date shifting with leap-year clamping
(`shared/calendar/academic-year.ts`).

Adding one year to a date is not simply incrementing its year component: calling `setUTCFullYear` on the 29th of February of a leap year silently rolls the date forward into the 1st of March whenever the target year is not itself a leap year, since the 29th of February does not exist there. The function detects this by checking whether the resulting month still matches the source month; if it does not, the date is rolled back with `setUTCDate(0)`, which JavaScript resolves to the last day of the previous month, that is, the 28th of February. Every other date is left unaffected, since shifting a day that exists in every year by whole years never changes its month.

4.7 Business Rule Engine

A modular rule engine validates event occurrences against configurable institutional constraints. Each rule is a self-contained unit implementing a shared interface (a `validate` function that receives the event, the occurrence, its configuration, and validation context such as vacation periods and other events); the engine itself has no knowledge of what a rule checks, it only calls this function and collects the result. Rules are registered in a single, central list that the engine iterates over, with no branching logic deciding which rule applies, so new institutional rules can be added by writing one new file and registering it, without touching the migration or validation logic. This same rule registry also drives the rule-configuration UI in the event form (Section 5.3.4), which reads each rule’s field metadata to render the correct input controls.

Code Example 4.4 shows one rule in full, the Weekend rule, which flags an occurrence whose start date, end date, or any date spanned by a multi-day occurrence, falls on a Saturday or Sunday.

```

1 export const weekendRule: IRuleDefinition = {
2   id: "weekend",
3   fields: [
4     { id: "checkStart", type: "checkbox", defaultValue: true },
5     { id: "checkEnd", type: "checkbox", defaultValue: true },
6     { id: "checkRange", type: "checkbox", defaultValue: false },
7   ],
8   translations: {
9     pt: {
10      label: "Fim de semana",
11      fields: { checkStart: "Verificar início", checkEnd: "Verificar fim", checkRange: "
12        Verificar período" },
13      fieldLabels: { start: "Início", end: "Fim", range: "Período" },
14      messages: {
15        startIsWeekend: "O início coincide com um fim de semana",
16        endIsWeekend: "O fim coincide com um fim de semana",
17        rangeHasWeekend: "O período inclui um dia de fim de semana",
18      },
19    },
20    validate(_event, occurrence, config) {
21      const { checkStart, checkEnd, checkRange } = config as WeekendConfig;
22      const violations: IRuleViolation[] = [];
23
24      if (checkStart && isWeekend(parseISO(occurrence.startDate))) {
25        violations.push({ fieldLabelKey: "start", date: occurrence.startDate, messageKey: "
26          startIsWeekend" });
27      }
28      if (checkEnd && isWeekend(parseISO(occurrence.endDate))) {
29        violations.push({ fieldLabelKey: "end", date: occurrence.endDate, messageKey: "
30          endIsWeekend" });
31      }
32      if (checkRange) {
33        for (const day of eachDayOfInterval({ start: parseISO(occurrence.startDate), end:
34          parseISO(occurrence.endDate) })) {
35          if (isWeekend(day)) {
36            violations.push({ fieldLabelKey: "range", date: day.toISOString(), messageKey: "
37              rangeHasWeekend" });
38            break;
39          }
40        }
41      }
42      return violations;
43    },
44  };

```

Code Example 4.4: The Weekend rule definition
(server/calendar/rules/definitions/weekend.rule.ts).

The id is the stable key stored in `event_rules.type` and used to dispatch validation; `fields` declares the rule's three independent toggles, checking the start date, the end date, and every date in between, together with their default values, which the event form (Section 5.3.4) reads to render the matching checkboxes with no rule-specific UI code required;

`translations` supplies every user-facing string the rule needs, from its display label to one message per kind of violation, keyed by locale; and `validate` is the pure function the engine calls with the event, one of its occurrences, and this rule’s configuration, returning zero or more violations. Each violation carries a message key and the specific field and date that triggered it, rather than a pre-formatted string, so the frontend can translate and render it consistently with every other rule (Section 5.3.5).

Seven rules are currently implemented:

- **Weekend:** flags occurrences whose start, end, or intermediate dates fall on a Saturday or Sunday.
- **Holiday:** flags occurrences that coincide with Portuguese national holidays.
- **Vacation period:** flags occurrences that overlap a registered school vacation period for that academic year.
- **Event no-overlap:** flags occurrences whose overall date span overlaps the span of other, user-selected events.
- **Minimum gap to event:** enforces a minimum number of working days between the end of one event and the start of a related, user-selected event.
- **Minimum duration:** an always-on, built-in rule enforcing a minimum calendar-day duration for an occurrence, driven by the occurrence’s own `minDays` field.
- **Minimum gap to next occurrence:** an always-on, built-in rule enforcing a minimum working-day gap between consecutive occurrences of the same event, driven by `minDaysToNext`.

The first five rules are selectively enabled and parameterized per event through the event creation and editing interface; the last two are always active for every occurrence and are not exposed in the rule-configuration UI (they are marked `hidden` in the rule metadata). All rules are advisory: none of them block the creation, editing, or migration of an event; violations are surfaced for manual review rather than enforced automatically. This business-rule layer is distinct from the structural, blocking validation performed by Zod on every API request (e.g. ensuring an occurrence’s end date is later than its start date).

4.8 Tests

An automated test suite, built with Vitest, now covers the system’s pure-logic layer: code with no database or network dependency, which is where the project’s business-critical logic is concentrated. This comprises the seven business rules and the rule dispatcher (Section 4.7),

the working-day calculation utility that two of those rules depend on, the password hashing and verification functions (Section 4.2), and the academic-year date helpers used by the Migration Engine (Section 4.6). The suite comprises 11 test files and 61 tests, all passing, reaching approximately 98% statement coverage on the files it targets, as measured by `@vitest/coverage-v8`.

Two choices keep the suite deterministic across machines and CI runs rather than dependent on the current date or the host’s timezone: the test process’s timezone is pinned to UTC, and test cases use fixed calendar dates, including real Portuguese public holidays and known weekend dates, instead of computing them relative to “today”.

The service layer, the repository/data-access layer (both the PostgreSQL and Redis implementations), API routes, and the React frontend are covered by manual testing: exercising each functional requirement’s flows (event and occurrence CRUD, the migration engine, rule validation, login, and user management) directly through the application’s UI and, for API endpoints, through direct HTTP requests.

The application was also tried by a small group of ISEL academic staff, the system’s intended end users, who used it on their own initiative rather than following a guided script. Their feedback, given informally in conversation and in writing, was generally positive and led to a number of user-interface adjustments.

Chapter 5

Frontend

This chapter describes the frontend implementation of EduCal: how the codebase is organized, how it communicates with the backend, and every screen and dialog that makes up the application.

5.1 Structural Organization

EduCal is a single Next.js application (App Router) that hosts both frontend and backend, rather than two separate projects. The `app/` directory is kept as a thin routing layer (the calendar home page, the login page, the admin users page, and the API route handlers under `app/api/`), while the actual UI is organized by feature under `features/`: `features/calendar/` (the calendar itself, including views, dialogs, header controls, drag-and-drop, and settings), `features/auth/` (the login form), and `features/admin-users/` (the user management dashboard). Reusable, feature-agnostic UI primitives (buttons, dialogs, forms, selects, etc.) live under `components/ui/`, and code shared between the client and the server (types, Zod schemas, role logic) lives under `shared/`.

The files under `components/ui/` come from `shadcn/ui`, which is not a conventional dependency: rather than installing a compiled package from `node_modules`, its CLI (`npm shadcn add <component>`) copies each component's source code directly into the project. This means the primitives living in `components/ui/` originate from that shared, open-source library and were generated and adapted, not authored from scratch, even though they sit alongside the project's own code with no external package boundary marking them as such.

5.2 API Communication Architecture

EduCal deliberately does not use HTTP API routes for reading data. Server components fetch data directly by calling backend services: `features/calendar/requests.ts`, marked "server-only", is invoked once from the `Calendar` server component, and the resulting data (events, users, enum values, vacations) is passed down as initial props into a React

context provider (**CalendarProvider**). This avoids an unnecessary network round-trip for the common case of simply loading the calendar.

Mutations (creating or editing an event, logging in, managing users) do go through HTTP: a shared fetch wrapper (`lib/api-client.ts`) centralizes this logic. It attaches the JSON content type, unwraps the `{ data: ... }` envelope on success, and on failure parses the backend's `{ error: { code, message, details } }` envelope (Chapter 4) into a JavaScript error carrying a user-facing, translated message. Feature-specific request modules (`features/calendar/client-requests.ts`, `features/admin-users/client-requests.ts`) build on top of this wrapper for each mutation endpoint. Components call these functions inside a try/catch block and surface the result as a toast notification (success or error) using **sonner**, giving the user immediate feedback without a full page reload.

5.3 Web Application

5.3.1 Navigation and Access Control

The application has three main routes: the calendar (`/`), the login page (`/login`), and the admin users page (`/admin/users`). Route-level access control is handled entirely on the server, in the page component itself, rather than by a client-side auth context: `app/page.tsx` resolves the current session and redirects to `/login` if none exists, before any calendar markup is ever sent to the client; `app/login/page.tsx` redirects an already-authenticated user straight to `/`; and `app/admin/users/page.tsx` redirects to `/login` if unauthenticated and to `/` if the current user is not an **admin**. On the calendar page, once the session check passes, the actual event/user/enum/vacation data is fetched inside the **Calendar** component behind a **Suspense** boundary, which shows a loading skeleton while it resolves.

5.3.2 Authentication

Figure 5.1 shows the login form, which has three fields: **email**, **password**, and a “Remember Me” checkbox (**staySignedIn**, checked by default) that controls how long the session cookie persists (Section 4.2). Fields are validated client-side with the same Zod schema shape used server-side, highlighting invalid fields in red. On submit, the button switches to a disabled “submitting...” state; on success the app navigates back to the calendar and refreshes its server components so the newly authenticated session is picked up immediately; on failure, the server’s error message is shown inline below the form, rather than as a toast.

The login interface is a white rounded rectangle with a light gray border. It contains the following elements from top to bottom:

- An "Email" label above a yellow input field containing the text "daniel@educal.local".
- A "Password" label above a yellow input field filled with black dots.
- A checkbox with a blue checkmark and the text "Manter sessão iniciada".
- A black button with the white text "Entrar".

Figure 5.1: Login interface.

5.3.3 Calendar Interface

The interactive calendar (Figure 5.2) is the core of the application. Its header is split into two rows: a top row with a “Today” shortcut and a date navigator (showing the current month/year, an event-count badge for the visible period, and previous/next controls), and an actions row containing the category filter, the Agenda/Day/Week/Month/Year view switcher, the academic-year tools (Section 5.3.5), a user filter, the “Add Event” button, and the settings menu.

The five views (**Agenda**, **Day**, **Week**, **Month**, **Year**) offer different levels of temporal density, from a scrollable list grouped by date or category (Agenda) to a full-year grid overview (Year). Within the day/week grid views, occurrences can be rescheduled directly by dragging them to a different time slot, or resized to change their duration, both implemented with the native HTML5 drag-and-drop API and animated with **framer-motion**; both actions persist immediately through the mutation flow described in Section 5.2. When a month-view day has more occurrences than can be shown inline, an “+N more” control opens a simple list dialog for that day, from which any occurrence can be opened in full detail.

Code Example 5.1 shows the **DraggableEvent** component that wraps every occurrence block shown in the day and week views, connecting the browser’s native drag events to the shared drag-and-drop context.

```

1 export function DraggableEvent({
2   event,
3   occurrence,
4   children,
5   className,
6 }: DraggableEventProps) {
7   const { canEditEvents } = useCalendar();
8   const { startDrag, endDrag, isDragging, draggedOccurrence } = useDragDrop();
9
10  if (!canEditEvents) {
11    return <>{children}</>;
12  }

```

```

13
14 const isCurrentlyDragged =
15   isDragging && draggedOccurrence?.occurrence.id === occurrence.id;
16
17 return (
18   <motion.div
19     className={` ${className} || ""} ${
20       isCurrentlyDragged ? "opacity-50 cursor-grabbing" : "cursor-grab"
21     }`
22     draggable
23     onClick={e => e.stopPropagation()}
24     onDragStart={e => {
25       (e as DragEvent).dataTransfer!.setData("text/plain", occurrence.id);
26       startDrag(event, occurrence);
27     }}
28     onDragEnd={() => endDrag()}
29   >
30     {children}
31   </motion.div>
32 );
33 }

```

Code Example 5.1: The `DraggableEvent` wrapper component
(`features/calendar/dnd/draggable-event.tsx`).

The component reads two pieces of shared state: `canEditEvents` from the calendar context, which it uses to fall back to rendering its children unwrapped, with no drag handlers attached at all, for users without editing rights; and `useDragDrop`, a context that tracks which single occurrence, across the whole calendar body, is currently being dragged. Setting the native `draggable` attribute on a `motion.div` lets `framer-motion` animate the wrapper, dimming it to 50% opacity while it is the one being dragged, while still relying on the browser's own HTML5 drag-and-drop events, `onDragStart` and `onDragEnd`, to notify the context when a drag begins and ends. The component only originates the drag: the drop target and the resulting date/time update are handled by the companion `DroppableArea` component and the `updateOccurrence` mutation (Section 5.2), keeping each of the two halves of the interaction in its own, independently reusable component.

Two controls narrow down what is shown: a category filter (multi-select dropdown, one entry per `Category`, each with its colored dot) and a user filter (single-select, showing every user's avatar and name, defaulting to "All users"). A settings menu offers independent toggles for dark/light theme, category badge style (colored block vs. small dot), 12/24-hour time format, and how the Agenda view groups events (by date or by category), all persisted locally per browser.

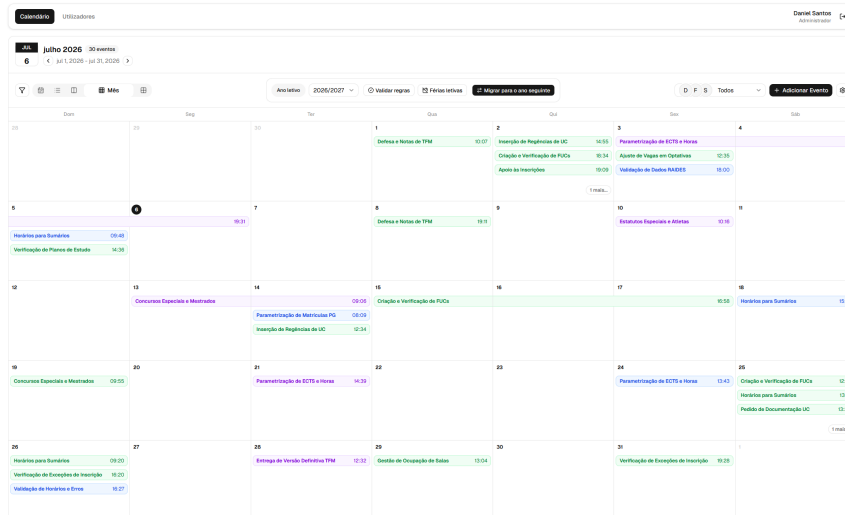


Figure 5.2: Main calendar interface (Month view).

5.3.4 Event Management Dialogs

Event and occurrence management is built around a small set of focused dialogs.

The **Add/Edit Event dialog** (Figure 5.3) is a single form reused for both creating and editing an event, visible only to users with editor-level access or higher. It is split into an event-level panel, with **name**, **objective**, and **category/classification/status/responsible** (each a dropdown populated from the corresponding lookup table), and an occurrence panel, where occurrences are added and removed freely as a repeatable list (at least one is always required), each with its own **description**, **start/end date-time**, and optional minimum-duration/minimum-gap-to-next-occurrence constraints. A dedicated “Rules” panel lets the user attach any of the five configurable business rules (Section 4.7) to the event from a dropdown; each added rule renders its own input controls generated from that rule’s field metadata, namely checkboxes, number inputs, or an event picker, depending on the rule, and can be removed independently. All fields are validated with Zod before submission, including a check that every occurrence’s end date is after its start date.

Figure 5.3: Add/Edit Event dialog.

The **Event Details dialog** is the read-only counterpart, opened when a user clicks an occurrence anywhere in the calendar. It shows the event’s name, category/classification/status badges, its objective, who created it, the selected occurrence’s dates, every occurrence in a scrollable grid, and a summary of the rules attached to the event, in the same terms an end user configured them in. Users with edit rights see shortcuts here to open the Edit dialog or delete the event directly.

Deleting an event goes through a dedicated **confirmation dialog** (a destructive-styled button that opens an alert requiring explicit confirmation) before the event and all of its occurrences are removed, since occurrence deletion cascades from the event at the database level (Section 3.3).

Figure 5.4: Event Details dialog.

5.3.5 Academic Year Tools

The academic-year tools, part of the calendar’s actions bar, let the user pick an academic year and run the migration workflow described in Section 4.6: a “Validate” action checks the selected year’s events against every business rule without changing anything, and a “Migrate” action (available to editors and administrators) runs the full migration from that year into the next. Both actions share a single results modal (Figure 5.5): if no rule violations were found, it shows a simple confirmation banner; otherwise, each violation is listed as its own card with the rule that was broken, the affected event and occurrence, a human-readable explanation, and the offending date, and clicking a card opens that event directly in the Event Details dialog. A separate “Vacations” action opens a dedicated dialog (Figure 5.6) for managing that academic year’s vacation periods (label and date range), with full create/edit/delete support for users with edit rights.



Figure 5.5: Validation/migration results modal.



Figure 5.6: Vacation periods dialog.

5.3.6 User Management Dashboard

The administrative dashboard (Figure 5.7), accessible only to administrators, lists every user in a table (name, email, role, profile picture, and actions) and supports full CRUD. Creating a user requires name, email, role, an optional profile picture path, and a password; editing reuses the same form inline below the table and only sends the fields that actually changed, with the password left blank meaning “keep the current password”. Every mutation gives immediate feedback via toast notifications, and deleting a user is confirmed before proceeding. The delete action is disabled for the currently logged-in administrator’s own row, reinforcing the “cannot delete self” invariant already enforced on the backend (Section 4.4).

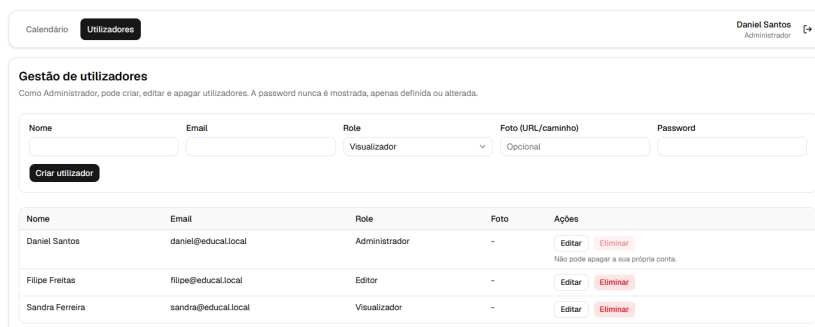


Figure 5.7: User management dashboard.

Chapter 6

Deployment and Infrastructure

6.1 Current Infrastructure

During development, EduCal was run and previewed using **Vercel**, the hosting platform built by the creators of Next.js. It requires minimal configuration for Next.js applications, which made it possible to get a working, publicly reachable preview of each iteration of the application without needing to provision any institutional infrastructure up front.

Provisioning a remote PostgreSQL server for this preview deployment was not warranted at this stage of the project, so the Vercel deployment instead runs on the JSON-blob-backed Redis implementation (`DATA_LAYER=redis`, Section 3.4), avoiding the need for a remote database server entirely. PostgreSQL itself was used and tested only against a local instance throughout development; the ISEL virtual machine described below is intended to be the first environment where it runs continuously.

Locally, the project is run with the standard Next.js toolchain, managed through `pnpm`: `pnpm dev` for the development server, and `pnpm build/pnpm start` to build and run a production Node.js server, with database migrations (`pnpm run db:migrate`) run manually against whichever backend is configured.

6.2 Deployment Roadmap

Because EduCal manages institutional academic planning data, the intended production deployment is a single virtual machine hosted within **ISEL's own internal network**, rather than a third-party cloud platform, so that both the application and its data remain under the institution's direct control. This VM would host both the Next.js application (built and run with `pnpm build/pnpm start`) and the PostgreSQL instance.

A single VM, rather than two separate machines for the application and the database, is the intended setup. At the scale EduCal is designed for, a single institution's academic staff, splitting the two across separate VMs would add operational overhead (two machines to provision, patch, and monitor instead of one) and network configuration (securing traffic between them) without a corresponding benefit: that split earns its complexity when the

application and database need to scale independently, for example running several application instances behind a load balancer against one shared database, which is not a requirement here. Colocating both keeps the surface ISEL's IT staff need to secure and maintain down to a single machine, at the cost of the application and database competing for the same CPU, memory, and disk, an acceptable trade-off given the expected load.

This step has not been carried out yet: no VM has been provisioned or configured as part of this project's delivery. Setting it up is expected to follow these steps:

1. Receive the VM from ISEL's IT department, provisioned with a Linux distribution, and confirm its network reachability (whether it is reachable only within ISEL's internal network, or also externally, which determines whether a reverse proxy/TLS setup is needed).
2. Install Node.js (the version this project targets), `pnpm`, and a PostgreSQL server on the VM.
3. Create the application's PostgreSQL database and role, and configure `DATABASE_URL` and the remaining environment variables in a `.env` file local to the VM, never committed to version control.
4. Run the Drizzle migrations (`pnpm run db:migrate`) against this new database to create the schema, followed by `pnpm run db:seed` to populate the reference lookup tables (categories, classifications, statuses, responsables) and create the initial administrator account.
5. Build the application (`pnpm build`) and run it as a persistent background service, for example via a process manager such as `pm2` or a `systemd` unit, so that it restarts automatically on crash or VM reboot.
6. Configure a reverse proxy (e.g. Nginx) in front of the Node.js process to handle TLS termination and serve the application on the standard HTTPS port, under whatever internal hostname ISEL assigns it.
7. Set up a recurring backup schedule for the PostgreSQL database (e.g. a nightly `pg_dump` to institutional storage), since this is the point where data loss becomes a real institutional risk once the calendar is in active use.

These steps, along with defining a repeatable deployment process for future updates, are the concrete next steps once the VM is made available by the institution.

Chapter 7

Conclusion

At the conclusion of the project timeline, EduCal has moved from a conceptual set of requirements to a functional web application that replaces the spreadsheet-based process previously used for institutional academic planning at ISEL (Chapter 2). Of the six functional requirements defined in Section 1.2.1, four were fully implemented and are in use: hierarchical event and occurrence management (RF01, Sections 3.3.1 and 5.3.4), the idempotent, rule-aware academic-year Migration Engine (RF02, Section 4.6), the configurable Business Rule Engine (RF03, Section 4.7), and role-based multi-user access control (RF06, Sections 4.2, 4.4, and 5.3.6). Two requirements from the original scope were not delivered: notification templates (RF04) and PDF schedule export (RF05); no corresponding functionality exists in the current codebase, and both are excluded from this delivery.

The decision to structure the backend as a decoupled Controller-Service-Data pipeline, with all storage access hidden behind a repository interface (Section 3.4), proved effective throughout the project. It allowed the migration engine and the rule engine to be built and iterated on independently of the underlying data store, and it made the transition from the JSON-backed mock data used while developing the business logic to the production PostgreSQL database a matter of changing a single environment variable, with no changes required to the controllers, services, API routes, or frontend. The same separation is what makes it possible to keep the lightweight Redis-backed implementation alongside PostgreSQL for development and demos, without either implementation leaking into the rest of the system.

Overall, EduCal addresses the core problem stated in Chapter 2, the manual, error-prone, spreadsheet-based rebuilding of the academic calendar every year, through automated migration and configurable rule checking, backed by a secure, role-based multi-user system. Completing notification templates and schedule export, and moving from the current local/Vercel setup to the planned ISEL VM deployment (Chapter 6), remain as the identified gaps between this delivery and the full original scope.

References

- [1] Vercel. Next.js documentation, 2026. [Online; accessed July 2026].
- [2] The PostgreSQL Global Development Group. Postgresql documentation, 2026. [Online; accessed July 2026].
- [3] OpenJS Foundation. Node.js crypto module documentation, 2026. [Online; accessed July 2026].
- [4] OWASP Foundation. Session management cheat sheet, 2026. [Online; accessed July 2026].